

ExosPlore

An End-to-End AI Pipeline for Detecting, Classifying, and Characterizing Exoplanet Transits in TESS Light Curves

1. The Idea, In Plain Words

When a planet passes in front of its star, the star's light dims by a tiny, repeating amount. TESS, a NASA space telescope, records the brightness of thousands of stars over time, producing what is called a light curve. Hidden inside these light curves are the faint dips caused by planets — but also dips caused by other things: orbiting pairs of stars, light from a neighbouring star bleeding into the same pixel, or starspots. The data is noisy, so it isn't always obvious what caused a dip.

ExosPlore is a software pipeline that takes a raw, noisy TESS light curve and automatically answers three questions:

1. Is there a repeating dip in this light curve at all?
2. If yes, what is most likely causing it — a planet, a binary star, a blended source, starspots, or just noise?
3. If it looks like a planet, what are its orbital period, transit duration, depth, and how confident are we?

The output is shown on a simple dashboard: the light curve, the detected dip, the predicted cause, the estimated planet parameters, and a confidence score — all in one view.

2. Why This Is Hard

A planet transit is not the only thing that can make a star look dimmer. Three other things create very similar-looking dips:

- Eclipsing binary stars — two stars orbiting each other, periodically blocking one another.
- Stellar blending — a second, unrelated star sits so close in the sky that its light contaminates the measurement.
- Starspots / stellar variability — dark spots on the star's own surface, rotating in and out of view.

On top of this, the raw data itself is noisy because of the detector's electronics and small pointing jitters of the spacecraft. A good pipeline must clean the noise without erasing the very signal it is looking for, and then correctly tell these four "look-alike" causes apart.

3. Our Approach At a Glance

ExosPlore is built as five connected stages. Each stage has one clear job, so the system stays easy to understand, debug, and explain to evaluators.

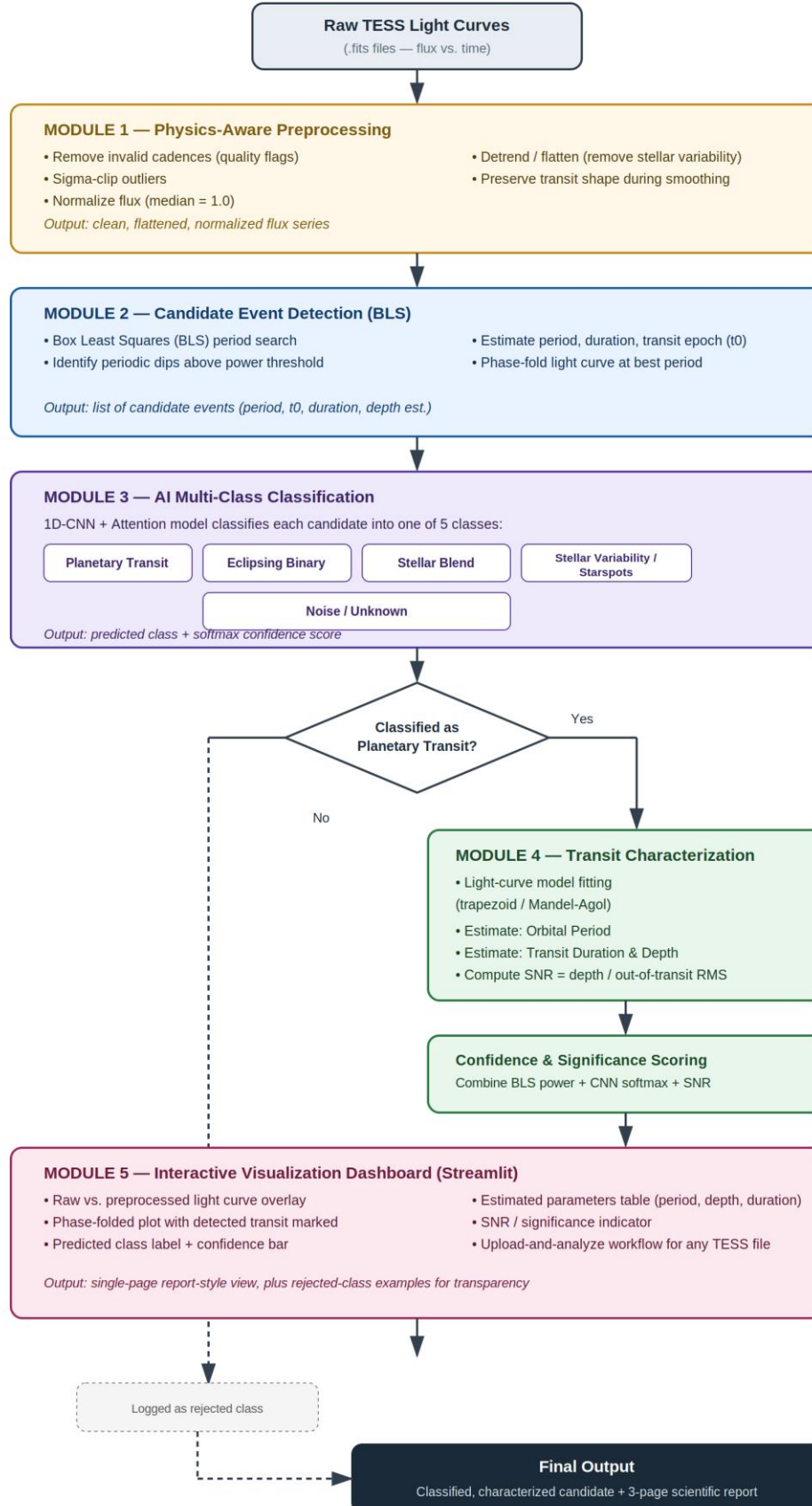
Stage	What it does	Output it passes on
1. Preprocessing	Cleans the raw light curve: removes bad data points, normalizes brightness, and removes slow stellar trends.	A clean, flattened light curve.
2. Candidate Detection	Searches for periodic dips using the Box Least Squares (BLS) algorithm — the standard tool astronomers use to find transit-like signals.	A short list of candidate events, each with an estimated period and timing.
3. Classification	A 1D Convolutional Neural Network with an attention layer looks at the shape of each candidate dip and decides what caused it.	A predicted class (planet / binary / blend / starspot / noise) with a confidence score.
4. Characterization	For anything classified as a planetary transit, fits a transit model to estimate the planet's orbital details and signal strength.	Orbital period, transit duration, transit depth, and signal-to-noise ratio (SNR).
5. Dashboard	Displays everything in one interactive view so a researcher can inspect, trust, and act on the result.	A visual report: light curve, prediction, parameters, and confidence.

4. System Architecture

The diagram below shows the full pipeline from raw input to final output, including how non-planet signals are still logged and shown for transparency rather than silently discarded.

ExosPlore — System Architecture

End-to-end pipeline: raw TESS light curve to classified, characterized exoplanet candidate



5. What Makes This Different From a Basic Solution

Many simple approaches to this problem train a model to answer only "planet or not planet." ExosPlore" goes further in ways that directly match what the problem statement asks for:

- Multi-class, not binary — the model explicitly separates binaries, blends, and starspots from real transits, instead of lumping them all into "not a planet."
- Physics-first, not just AI — the pipeline uses BLS, the established astronomical method for transit search, before any neural network is involved. The AI's job is classification, not raw detection.
- Every detection comes with numbers — period, duration, depth, SNR, and confidence — not just a label.
- Rejected detections are not hidden — eclipsing binaries and blends that the system correctly identifies and excludes are shown, demonstrating that the pipeline understands the difference, not just got lucky.
- Built to be explained — every stage produces an inspectable, visual output, making it possible to show exactly why a decision was made.

6. Tools and Technologies

Category	Tools
Language	Python
Data handling	NumPy, SciPy, Pandas
Astronomy	Astropy, Lightcurve (light curve I/O, BLS, preprocessing)
Machine learning	Scikit-learn (utilities, metrics), PyTorch (1D-CNN + Attention model)
Visualization	Matplotlib, Plotly
Dashboard	Streamlit
Version control	Git & GitHub

7. Data Sources

- Raw TESS light curves downloaded from the MAST archive: archive.stsci.edu/tess/tic_ctl.html — using one sector's high-cadence data (~20–30k light curves).
- A curated, labelled dataset (provided by the organizers) containing confirmed exoplanets, false positives, and eclipsing binaries, used to train and validate the classifier.

8. Expected Deliverables

- A working end-to-end pipeline: raw light curve in, classified and characterized result out.
- A trained multi-class classifier with reported accuracy per class.
- Estimated orbital period, transit duration, and transit depth for detected planet candidates.
- Signal-to-noise ratio and confidence score for every detection.
- An interactive Streamlit dashboard for visual inspection of results.
- A 3-page report describing methodology, assumptions, and how uncertainty was estimated.
- Complete, documented source code on GitHub.

9. In Summary

ExosPlore takes the noisy, ambiguous problem of finding planets in starlight and breaks it into five honest, explainable steps: clean the data, find the dips, classify what caused them, measure the planet if there is one, and show the result clearly. It is built specifically to do what the problem statement asks — multi-class detection, parameter estimation, confidence scoring, and visualization — rather than a shortcut binary classifier, while staying realistic to build within the hackathon's 30-hour window.